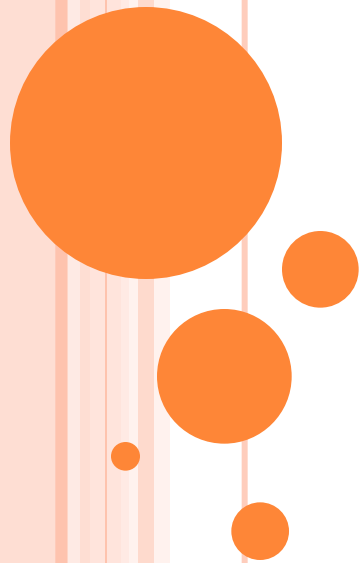


RESOLUTION PROOF METHOD

IN PROPOSITIONAL LOGIC



DECISION PROBLEMS IN PROPOSITIONAL/PREDICATE LOGIC

1. Is a propositional/predicate formula a tautology/theorem?

$$\overset{?}{\models} V \quad \text{or} \quad \overset{?}{\vdash} V$$

2. Is a propositional/predicate formula a logical/syntactic consequence of a set of hypotheses?

$$U_1, \dots, U_n \overset{?}{\models} V \quad \text{or} \quad U_1, \dots, U_n \overset{?}{\vdash} V$$

In order to solve these two decision problems, **proof methods** are applied.



RESOLUTION PROOF METHOD

- It was proposed by J.A. Robinson in 1965 as a proof method for classical logics
- Dedicated theorem provers based on resolution:
OTTER, PCPROOVE, AMPHION, Jape
- It was easily adapted to *nonstandard logics* (modal, temporal, many-valued, non-monotonic).
- Its basic aim is to check the ***consistency/inconsistency*** of a set of clauses.
- It is based on syntactic considerations => ***syntactic method***
- The ***validity*** of a formula **is proved by contradiction** => ***refutation method***



RESOLUTION METHOD

- FORMAL SYSTEM FOR PROPOSITIONAL LOGIC -

$Res = (\Sigma_{Res}, F_{Res}, A_{Res}, R_{Res})$, where:

- $\Sigma_{Res} = \Sigma_P - \{\rightarrow, \leftrightarrow, \wedge\}$ - the alphabet;
- $F_{Res} \cup \{\square\}$
 - F_{Res} - the set of all clauses built using the alphabet Σ_{Res} ;
 - $\square$ is the empty clause, does not contain any literal, it symbolizes inconsistency;
- $A_{Res} = \emptyset$ is the set of axioms ;
- R_{Res} is the set of inference rules containing the *resolution rule (res)*:
 $f \vee l, g \vee \neg l \vdash_{res} f \vee g$, where l is a literal and $f, g \in F_{Res}$.

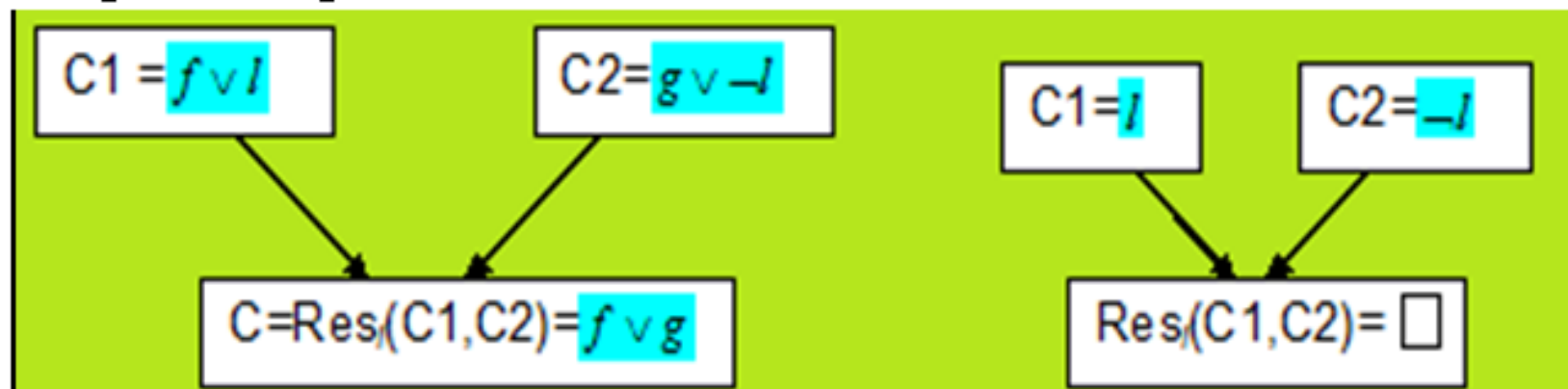


DEFINITIONS

Let l be a literal and $f, g \in F_{\text{Res}}$.

- the clauses $C_1 = f \vee l$ and $C_2 = g \vee \neg l$ are called *clashing clauses* and they *resolve upon the literal l* .
- notation: $C = \text{Res}_l(C_1, C_2) = f \vee g$, where C is called the *resolvent* of the *parent clauses* C_1 and C_2 .
- if $C_1 = l$ and $C_2 = \neg l$, then $\text{Res}_l(C_1, C_2) = \square$ (empty clause) which is inconsistent.

Graphical representation:



ALGORITHM: GENERAL_PROPOSITIONAL_RESOLUTION

input: S – a set of propositional clauses

output: message “ S is *consistent*” or “ S is *inconsistent*”

begin

$S_0 := S$; $i := 0$;

do

 choose two clashing clauses: $C_1, C_2 \in S_i$;

$C := \text{Res}(C_1, C_2)$;

$S_{i+1} := S_i \cup \{C\}$;

if ($C = \square$) **then** write “ S is *inconsistent*”; **exit**;

else $i := i + 1$;

end_if

until ($S_i = S_{i-1}$) // no new clauses can be derived

 write “ S is *consistent*”;

end



THEORETICAL RESULTS

Soundness theorem

If the empty clause is derived from the set S of propositional clauses using the resolution algorithm ($S \vdash_{\text{Res}} \square$), then S is an inconsistent set.

Completeness theorem

If the set S of propositional clauses is inconsistent, then the empty clause can be derived from S using the resolution algorithm ($S \vdash_{\text{Res}} \square$).

Soundness and completeness theorem

A set S of propositional clauses is inconsistent if and only if $S \vdash_{\text{Res}} \square$.



RESOLUTION - A REFUTATION PROOF METHOD

Theorem

A propositional formula U is a **theorem** (tautology) *if and only if* the empty clause can be derived from the conjunctive normal form of $\neg U$, using the resolution algorithm.

U is a theorem (tautology) *if and only if* $\text{CNF}(\neg U) \vdash_{\text{Res}} \square$.

Theorem

Let $U_1, U_2, \dots, U_n, V$ be propositional formulas.

$U_1, U_2, \dots, U_n \vdash V$ *if and only if*

$U_1, U_2, \dots, U_n \models V$ *if and only if*

$\text{CNF}(U_1 \wedge U_2 \wedge \dots \wedge U_n \wedge \neg V) \vdash_{\text{Res}} \square$.



EXAMPLE 1

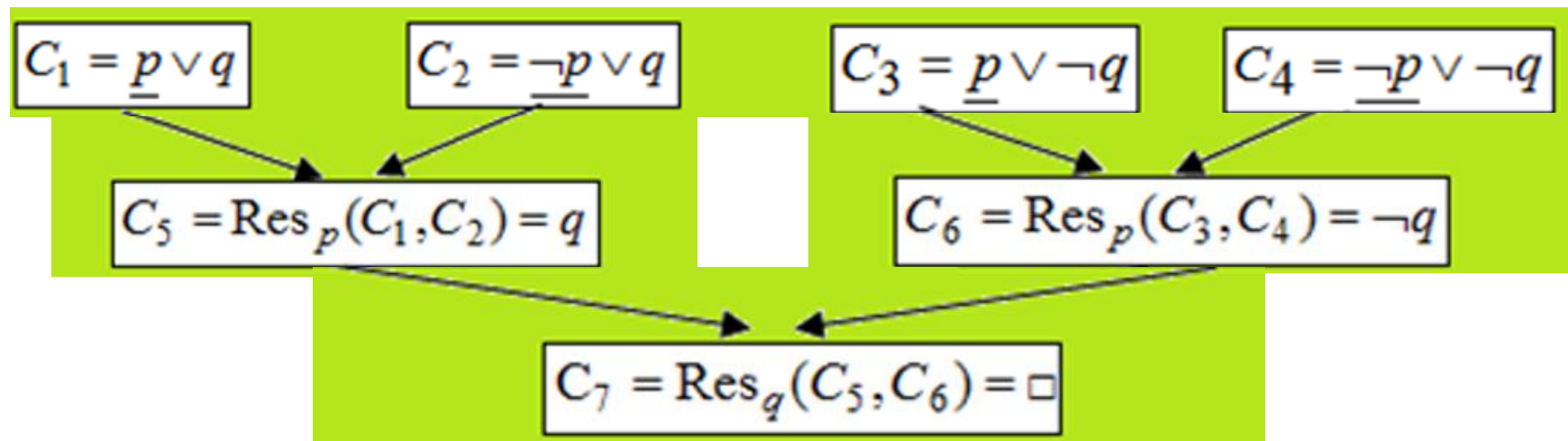
Using the resolution method prove that the set S of clauses is inconsistent.

$$S = \{p \vee q, \neg p \vee q, p \vee \neg q, \neg p \vee \neg q\}$$

We denote the propositional clauses as follows:

$$C_1 = p \vee q, \quad C_2 = \neg p \vee q, \quad C_3 = p \vee \neg q, \quad C_4 = \neg p \vee \neg q.$$

The process of deriving the empty clause is symbolized by the binary tree:



$S \vdash_{\text{Res}} \square$ and according to the *soundness theorem* we conclude that the set S of propositional clauses is inconsistent.

THEOREM

(BASED ON DAVIS-PUTMAN PROCEDURE)

A set S of propositional clauses can be simplified, preserving its **consistency/inconsistency**, by applying the following transformations:

- *Delete the clauses that are tautologies* because they are not useful in the derivation of the empty clause (from **true** we cannot derive **false**).
- *Delete the clauses subsumed by other clauses of S* (based on absorption).
 C_1 is subsumed by C_2 and C_2 subsumes C_1 if there exists a clause C_3 such that $C_1 = C_2 \vee C_3$.
- *Delete every clause that contains a pure literal.*
A *pure literal* in the set S of clauses is a literal that appears in a clause of S , but its negation does not appear in any clause of S .



EXAMPLE 2. MODELING REASONING

$H_1, H_2, H_3 \vdash^? C$

H_1 . If it is sunny, Diane and Alice go to the swimming pool.

H_2 . Ben goes to the swimming pool on Thursdays.

H_3 . It was sunny last Thursday.

C . Did Diane meet Ben at the swimming pool last Thursday?

$\text{CNF}(H_1 \wedge H_2 \wedge H_3 \wedge \neg C) \vdash_{\text{Res}}^? \square$

$H_1 : S \rightarrow D \wedge A \equiv \neg S \vee (D \wedge A) \equiv$
 $\equiv (\neg S \vee D) \wedge (\neg S \vee A) : C_1 \wedge C_2$

$H_2 : Th \rightarrow B \equiv \neg Th \vee B : C_3$

$H_3 : S \wedge Th : C_4 \wedge C_5$

$C : Th \wedge D \wedge B$

$\neg C : \neg(Th \wedge D \wedge B) \equiv \neg Th \vee \neg D \vee \neg B : C_6$

$X = \{C_1, C_2, C_3, C_4, C_5, C_6\}$

Notations for the propositional variables:

D – Diane goes to the swimming pool

A – Alice goes to the swimming pool

B – Ben goes to the swimming pool

S – it is sunny

Th – it is Thursday



EXAMPLE 2 – GENERAL RESOLUTION(CONTD.)

$$X = \{C_1, C_2, C_3, C_4, C_5, C_6\}$$

$$C_1 = \neg S \vee D$$

$$C_2 = \neg S \vee A$$

$$C_3 = \neg Th \vee B$$

$$C_4 = S$$

$$C_5 = Th$$

$$C_6 = \neg Th \vee \neg D \vee \neg B$$

The clause C_2 is eliminated because A is a pure literal in the set X and we obtain X' .

The refutation from X' (the derivation of $\square$ from X') is provided below:

$$C_7 = \text{Res}_{Th}(C_5, C_6) = \neg D \vee \neg B$$

$$C_8 = \text{Res}_B(C_7, C_3) = \neg D \vee \neg Th$$

$$C_9 = \text{Res}_{Th}(C_8, C_5) = \neg D$$

$$C_{10} = \text{Res}_D(C_9, C_1) = \neg S$$

$$C_{11} = \text{Res}_S(C_{10}, C_4) = \square$$

$\text{CNF}(H_1 \wedge H_2 \wedge H_3 \wedge \neg C) \vdash_{\text{Res}} \square$, so C is deducible from the hypotheses, therefore:

“Diane met Ben at the swimming pool last Thursday”.



IMPORTANT!

- To prove the **inconsistency** of a set of clauses **it is sufficient** to find a derivation of the empty clause.
- To prove the **consistency** of a set of clauses, a strategy for a complete search of the derivation of the empty clause **is necessary**.

Consistency is proved after generating all possible resolvents without deriving the empty clause.



STRATEGIES AND REFINEMENTS OF RESOLUTION

○ Strategies:

- assure that all the possible clauses to be derived are generated
- try to avoid the derivation of redundant and irrelevant clauses in order to obtain the empty clause.
- **level-saturation** strategy, **deletion** strategy, **set-of-support** strategy

○ Refinements

- make the resolution process more efficient by imposing restrictions on the clashing clauses
- **lock** resolution, **linear** resolution, **semantic** resolution



REMARKS

- All the refinements and strategies of resolution preserve the soundness and the completeness properties.
- All the combinations of these refinements and strategies preserve the soundness property.
- The completeness is not preserved in some combinations.
- *Incompleteness*: the initial set of clauses is inconsistent, but the empty clause cannot be derived because there are too many restrictions imposed by those strategies and refinements.

general resolution + deletion strategy is sound and complete

general resolution + set-of-support strategy is sound and complete

general resolution + deletion strategy + set-of-support strategy is sound
and complete



STRATEGIES OF RESOLUTION

- The *level saturation strategy* generates levels of resolvents corresponding to the exploration of the whole search space which contains all the possible resolvents.
- The *deletion strategy*: the resolvents that are tautologies or are subsumed by other clauses in the set S of clauses are eliminated and they will not be used further in the resolution process because they produce redundant clauses.



ALGORITHM: LEVEL-SATURATION-STRATEGY

input: S - the initial set of clauses;

output: message “ S is inconsistent” or “ S is consistent”

begin

// we generate the sequence $S^0, S^1, \dots, S^k$ containing sets of clauses (levels)

$S^0 := S$; $k := 0$; // initial level

do

{ $k := k + 1$;

$S^k := \{\text{Res}(C_i, C_j) \mid C_i \in S^{k-1}, C_j \in S^0 \cup S^1 \cup \dots \cup S^{k-1}\}$;

// we eliminate the resolvents obtained in the current level but
// which appear already in the previous levels

$S^k := S^k \setminus (S^0 \cup S^1 \cup \dots \cup S^{k-1})$;

} **until** ($\square \in S^k$ **or** $S^k = \emptyset$).

if ($\square \in S^k$)

then write “ S is inconsistent”

else // complete search for the derivation of $\square$

// no more resolvents can be derived and $\square \notin S^k$

write “ S is consistent”

end_if

end



EXAMPLE 3

Check the consistency/inconsistency of the set $S = \{p \vee q, \neg p \vee q \vee \neg r, \neg q \vee r\}$.

- level saturation strategy + deletion strategy
- the sequence $S^0, S^1, S^2, \dots$ represents levels of resolvents

$$S^k := \{\text{Res}(C_i, C_j) \mid C_i \in S^{k-1}, C_j \in S^0 \cup S^1 \cup \dots \cup S^{k-1}\}, k=1,2,\dots$$
- initial level: $S^0 = S = \{C_1 = p \vee q, C_2 = \neg p \vee q \vee \neg r, C_3 = \neg q \vee r\}$

First level

$$C_4 = \text{Res}_p(C_1, C_2) = q \vee \neg r$$

$$C_5 = \text{Res}_q(C_1, C_3) = p \vee r$$

$$C_6 = \text{Res}_q(C_2, C_3) = \neg p \vee \neg r \vee r \equiv T \text{ (tautology)}$$

$$C_7 = \text{Res}_r(C_2, C_3) = \neg p \vee q \vee \neg q \equiv T \text{ (tautology)}$$

C_6 and C_7 are not included in the first level

$$S^1 = \{C_4 = q \vee \neg r, C_5 = p \vee r\}$$

Second level

$$C_8 = \text{Res}_r(C_4, C_5) = q \vee p = C_1$$

$$C_9 = \text{Res}_q(C_4, C_3) = r \vee \neg r \equiv T$$

$$C_{10} = \text{Res}_r(C_4, C_3) = q \vee \neg q \equiv T$$

$$C_{11} = \text{Res}_p(C_5, C_2) = q \vee r \vee \neg r \equiv T$$

$$C_{12} = \text{Res}_r(C_5, C_2) = \neg p \vee p \vee q \equiv T$$

$$S^2 = \emptyset$$

$S^2 = \emptyset$ with the meaning that no more resolvents can be generated. *After a complete search, the empty clause was not derived from S and thus S is a consistent set.*

LOCK RESOLUTION

- This refinement of resolution was introduced by R.S. Boyer in 1971.
- It is very efficient and easily to be implemented.
- Each occurrence of a literal from a set of clauses is arbitrarily indexed with an integer.
- *Restriction*: the literals resolved upon must have the lowest indices in their clauses.
- The literals from resolvents inherit the indices from their parent clauses.
If the parent clauses have a common literal, in the resolvent this literal will have the lowest index from the inherited indices.
- At the implementation level we must combine lock resolution with the level saturation strategy in order to check all the possible ways of deriving $\square$.



THEOREMS – LOCK RESOLUTION

Soundness:

Let S be a set of clauses having each literal *arbitrarily* indexed with an integer.

If from S the empty clause can be derived using lock resolution ($S \vdash_{\text{Res}}^{\text{lock}} \square$),
then S is *inconsistent*.

Completeness:

Let S be a set of clauses having each literal *arbitrarily* indexed with an integer.

If S is *inconsistent*, then there is a lock derivation of the empty clause from S ($S \vdash_{\text{Res}}^{\text{lock}} \square$).

Note: The *completeness* property is not preserved if lock resolution is combined with the deletion strategy or with the set-of-support strategy.



REMARKS

- In order to prove the consistency of a set S of clauses, **lock resolution must be combined with the level saturation strategy:**

if $S^k = \emptyset$ (the last level of lock resolvents is empty)

then the set S is consistent.

- In order to prove **inconsistency** of a set S of clauses we do not need to use a **strategy**, it is enough to find a lock derivation of $\square$ from S .

If we combine lock resolution with the level saturation strategy the decision is:

if $\square \in S^k$ ($\square$ was derived as a lock resolvent in the k level)

then the set S is inconsistent.



EXAMPLE 4

Check the consistency/inconsistency of $S = \{\neg r, p \vee \neg q, r \vee \neg p, q\}$

Indexing of the literals:

$$C_1=(1)\neg r, C_2=(3)p \vee (2)\neg q, C_3=(4)r \vee (5)\neg p, C_4=(6)q$$

- lock resolution + level saturation strategy

$$S^0 = S$$

$$S^1 = \{\text{Res}^{\text{lock}}(C_i, C_j) \mid C_i \in S^0, C_j \in S^0\}$$

$$C_5 = \text{Res}_r^{\text{lock}}(C_1, C_3) = (5)\neg p$$

$$C_6 = \text{Res}_q^{\text{lock}}(C_2, C_4) = (3)p$$

$$S^1 = \{C_5, C_6\}$$

$$S^2 = \{\text{Res}^{\text{lock}}(C_i, C_j) \mid C_i \in S^1, C_j \in S^0 \cup S^1\}$$

$$C_7 = \text{Res}_p^{\text{lock}}(C_5, C_6) = \square$$

$$S^2 = \{C_7\}, \square \in S^2 \Rightarrow S \text{ is inconsistent}$$

Another indexing of the literals from clauses

$$C_1=(1)\neg r, C_2=(2)p \vee (3)\neg q, C_3=(5)r \vee (4)\neg p, C_4=(6)q$$

- lock resolution
- no strategy

The derivation of the empty clause:

$$C'_5 = \text{Res}_p^{\text{lock}}(C_2, C_3) = (3)\neg q \vee (5)r$$

$$C'_6 = \text{Res}_q^{\text{lock}}(C_4, C'_5) = (5)r$$

$$C'_7 = \text{Res}_r^{\text{lock}}(C_1, C'_6) = \square.$$

$\Rightarrow S$ is *inconsistent*

EXAMPLE 5

Using lock resolution and the level saturation strategy check the consistency/inconsistency of the following set of clauses:

$$S = \{p \vee q, p \vee \neg q \vee \neg r, \neg p \vee r\}.$$

The literals from the clauses are indexed as follows:

$$C_1 = {}_{(1)}p \vee {}_{(2)}q,$$

$$C_2 = {}_{(3)}p \vee {}_{(4)}\neg q \vee {}_{(5)}\neg r,$$

$$C_3 = {}_{(7)}\neg p \vee {}_{(6)}r,$$

$$S^0 = S = \{C_1, C_2, C_3\} \text{ - the initial set of clauses}$$

No lock resolvents can be generated ($S^1 = \emptyset$), the empty clause cannot be derived from S and we conclude that *the set S is consistent*.

EXAMPLE 6

Prove that *lock resolution + the deletion strategy is not complete* using the inconsistent set $S = \{p \vee q, \neg p \vee q, p \vee \neg q, \neg p \vee \neg q\}$ and the indexing:

$$C_1 = {}_{(2)}p \vee {}_{(1)}q, C_2 = {}_{(3)}\neg p \vee {}_{(4)}q, C_3 = {}_{(5)}p \vee {}_{(6)}\neg q, C_4 = {}_{(8)}\neg p \vee {}_{(7)}\neg q$$

Using the restriction imposed by lock resolution we can derive only the resolvents C_5 and C_6 which are tautologies:

$$C_5 = \text{Res}_p^{\text{lock}}(C_1, C_4) = {}_{(2)}p \vee {}_{(8)}\neg p \quad \text{and} \quad C_6 = \text{Res}_p^{\text{lock}}(C_2, C_3) = {}_{(4)}q \vee {}_{(6)}\neg q$$

If we apply also the deletion strategy, and we delete these tautologies, then we cannot continue the derivation process to obtain $\square$.

Note that even if S is inconsistent, the empty clause ($\square$) cannot be derived because there are too many restrictions imposed by lock resolution and the deletion strategy.

Therefore, *lock resolution + the deletion strategy is not complete*.



EXAMPLE 6 (CONTD.)

$$C_1 = {}_{(2)}p \vee {}_{(1)}q, \quad C_2 = {}_{(3)}\neg p \vee {}_{(4)}q, \quad C_3 = {}_{(5)}p \vee {}_{(6)}\neg q, \quad C_4 = {}_{(8)}\neg p \vee {}_{(7)}\neg q$$

$$C_5 = \text{Res}_p^{\text{lock}}(C_1, C_4) = {}_{(2)}p \vee {}_{(8)}\neg p \quad \text{and} \quad C_6 = \text{Res}_p^{\text{lock}}(C_2, C_3) = {}_{(4)}q \vee {}_{(6)}\neg q \quad - \text{tautologies}$$

Continuing the lock resolution process but using also the resolvents C_5 and C_6 we obtain:

$$C_7 = \text{Res}_p^{\text{lock}}(C_2, C_5) = {}_{(4)}q \vee {}_{(8)}\neg p$$

C_7 and C_2 are similar but not identical, the literals order (provided by the indices) differs in these clauses:

$\neg p$ is the literal to resolve upon from C_2 , and q is the literal to resolve upon from C_7 .

Thus, the role of a **tautology** as a parent clause is to modify the literals order in the other parent clause.

The lock resolution process is not blocked anymore and we use C_7 .

The following resolvents are obtained:

$$C_8 = \text{Res}_q^{\text{lock}}(C_7, C_4) = {}_{(8)}\neg p,$$

$$C_9 = \text{Res}_p^{\text{lock}}(C_3, C_8) = {}_{(6)}\neg q,$$

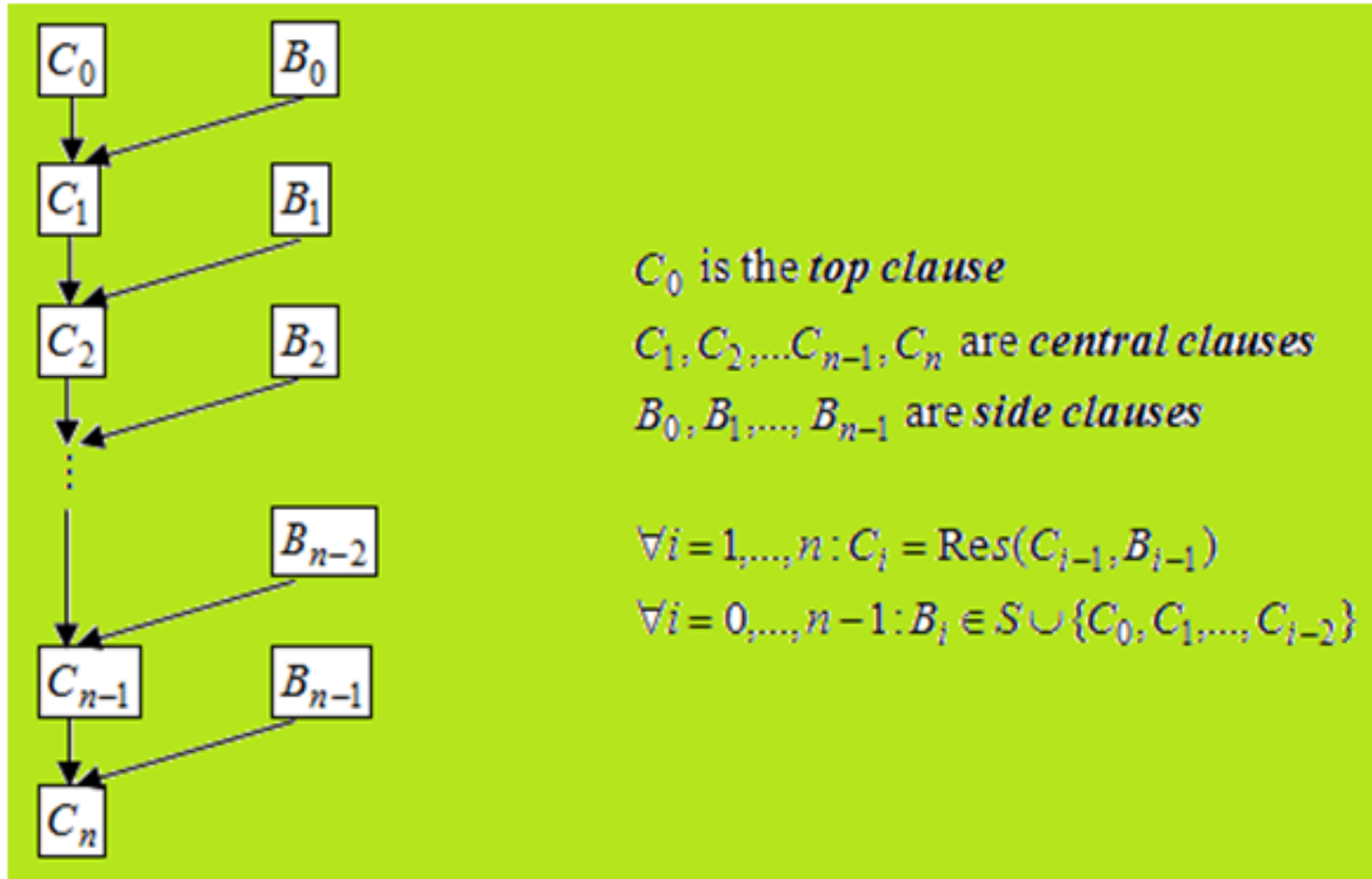
$$C_{10} = \text{Res}_q^{\text{lock}}(C_1, C_9) = {}_{(2)}p,$$

$$C_{11} = \text{Res}_p^{\text{lock}}(C_{10}, C_8) = \square.$$

As a conclusion, in this indexing the empty clause was derived from S with the help of **tautologies**.

LINEAR RESOLUTION (LOVELAND 1971)

For a set S of clauses, a *linear deduction* of C_n from S with $C_0 \in S$ as the top clause is symbolized graphically as follows:



THEORETICAL RESULTS

Soundness and completeness theorem

The set S of clauses is inconsistent if and only if $S \vdash_{\text{Res}}^{\text{lin}} \square$.

We can combine the linear resolution with the deletion strategy and the completeness property is preserved.

This refinement of resolution also provides a strategy at the implementation level: **backtracking** algorithm.

- The **inconsistency** of a set of clauses is proved when the empty clause is derived.
- The **consistency** of a set of clauses is proved after a complete search without the derivation of the empty clause.



Backtracking algorithm:

In each iteration, for the current central clause there are more possible side clauses. We continue the resolution process choosing one side clause. If in the iteration i the process is blocked (the central clause C_i is a tautology or it is an existing central clause: $C_i = C_j, j < i$, or C_i does not have side clauses) or all the side clauses of C_i were used, then we go back to the previous iteration ($i-1$) and we choose another possible side clause for C_{i-1} to continue the resolution process.

The algorithm stops in two cases:

- the empty clause was derived and the conclusion is that S is inconsistent.
- for the top clause all the possible side clauses were used, but the empty clause was not derived, then we conclude that the set S is consistent.

The consistency of a set of clauses is proved after a complete search without the derivation of the empty clause.



SPECIAL CASES OF LINEAR RESOLUTION

- *unit resolution*:
 - the central clauses have at least a unit clause as a parent clause.
- *input resolution*:
 - all side clauses are initial (input) clauses.

Theorem (equivalence of *unit resolution* and *input resolution*)

Let S be a set of propositional clauses. $S \vdash_{\text{Res}}^{\text{input}} \square$ *if and only if* $S \vdash_{\text{Res}}^{\text{unit}} \square$.

These two refinements of resolution are **sound**, but they are **not complete**:

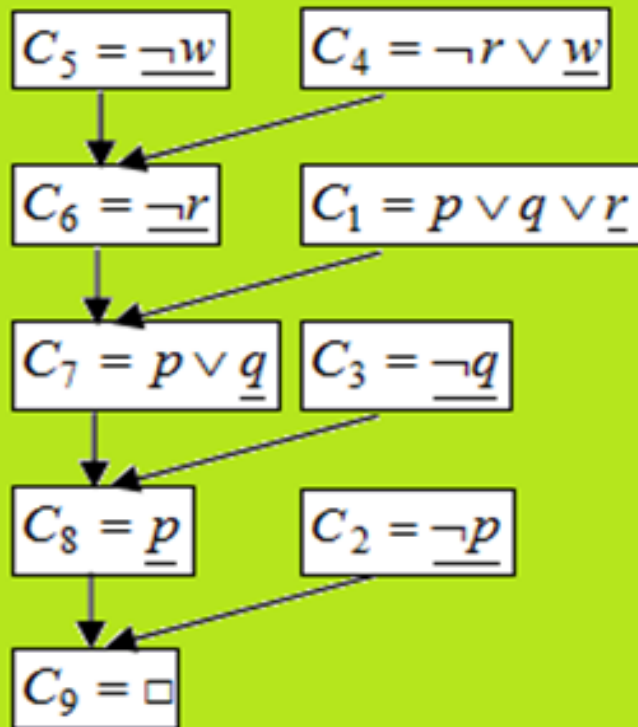
- **soundness**: If $S \vdash_{\text{Res}}^{\text{input/unit}} \square$ then S is inconsistent;
- **incompleteness**: there exist inconsistent sets of clauses from which the empty clause cannot be derived using input or unit resolution.



EXAMPLE 8

Prove the inconsistency of the set S of clauses: $S = \{C_1, C_2, C_3, C_4, C_5\}$.

$$C_1 = p \vee q \vee r, \quad C_2 = \neg p, \quad C_3 = \neg q, \quad C_4 = \neg r \vee w, \quad C_5 = \neg w$$



- This linear refutation is also a **unit** and an **input** refutation
- $S \vdash_{\text{Res}}^{\text{lin}} \square$, therefore S is an *inconsistent set*.

Example 7. Check the consistency/inconsistency of $S = \{C_1 = p \vee q, C_2 = \neg p \vee q, C_3 = \neg p \vee \neg q\}$

The linear search of the derivation of $\square$ using the backtracking strategy:

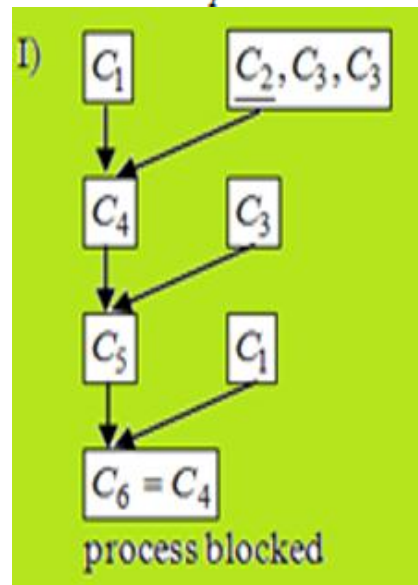
C_1 : **top clause**

I) C_2 is used as a side clause for C_1

$$C_4 = \text{Res}_p(C_1, C_2) = q$$

$$C_5 = \text{Res}_q(C_4, C_3) = \neg p$$

$$C_6 = \text{Res}_p(C_5, C_1) = q = C_4. \text{ Process blocked.}$$



II) C_3 is used as a side clause for C_1

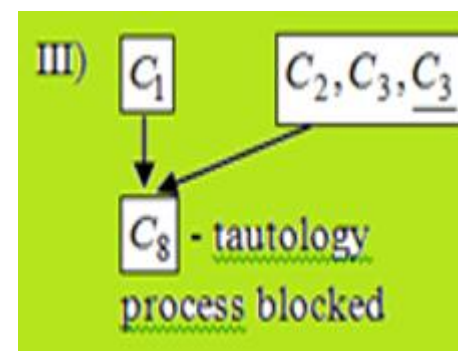
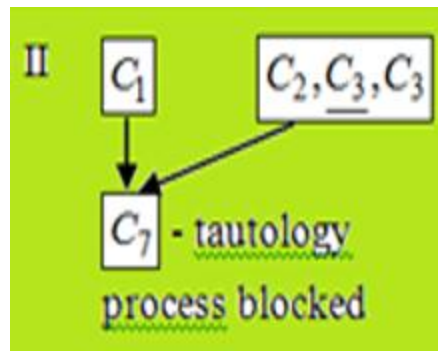
$$C_7 = \text{Res}_p(C_1, C_3) = q \vee \neg q \equiv T \text{ (tautology)}$$

The search process is blocked.

III) C_3 is used as a side clause for C_1

$$C_8 = \text{Res}_q(C_1, C_3) = p \vee \neg p \equiv T \text{ (tautology)}$$

The search process is blocked.



A complete linear derivation search was performed, but $\square$ was not derived, so S is a **consistent** set.